

Software Analysis Report of SCMS		
Doc # SCMS-SAR	Version: 1.1	Page 1 / 9

REVISION HISTORY

Date	Version	Description	Author
08/05/2026	1.0	Initial draft created; Introduction and Analysis Methodology defined.	Emir, Bora
09/05/2026	1.1	Static Code Analysis (SpotBugs) integrated; Results and Issue Table added.	Hamza
10/05/2026	1.2	Dependency Analysis (JDepend) completed; Architecture mapping finalized.	Genidy
11/05/2026	1.3	Test Coverage Analysis (JaCoCo) integrated; Final review and polishing.	All Members

Software Analysis Report of SCMS		
Doc # SCMS-SAR	Version: 1.1	Page 2 / 9

TABLE OF CONTENTS

Revision History

	Error! Bookmark not defined.	
1	Introduction	2
1.1	<i>Document overview</i>	Error! Bookmark not defined.
2	Static Code Analysis	3
2.1	<i>Tools</i>	4
2.2	<i>Results and Discussion</i>	Error! Bookmark not defined.
3	Dependency Analysis	
	Error! Bookmark not defined.	
3.1	<i>Tools</i>	6
3.2	<i>Results and Discussion</i>	Error! Bookmark not defined.
4	Test Coverage Analysis	7
4.1	<i>Tools</i>	8
4.2	<i>Results and Discussion</i>	Error! Bookmark not defined.

Software Analysis Report of SCMS		
Doc # SCMS-SAR	Version: 1.1	Page 3 / 9

1 Introduction

1.1 Document overview

This document presents and interprets the code analysis results regarding the SCMS (Student Club Management System) software development project. Code is analyzed by 3 different tools:

1. Static Code Analysis tool
2. Dependency Analysis tool
3. Test Coverage tool

The first tool is used to reveal potential bugs that might be overseen during the testing process. The second tool is employed for evaluating the design quality based on the amount of coupling among the software modules and to what extent the code reflects the originally envisioned 3-tier design (Presentation, Application, Data). The last tool is used for measuring the coverage of unit tests in the project, particularly focusing on the `scms.application` package where the core business logic resides. Each section below is dedicated to each of these 3 analyses.

Software Analysis Report of SCMS		
Doc # SCMS-SAR	Version: 1.1	Page 4 / 9

2 Static Code Analysis

2.1 Tools

For the static code analysis of the SCMS project, we used SpotBugs and CheckStyle. SpotBugs was run via the IntelliJ IDEA plugin to inspect the compiled bytecode for potential bugs, bad practices, and performance issues. CheckStyle was configured against the Official Oracle Java Code Conventions referenced in our Software Development Plan (SDP) so that any deviation in naming, indentation or brace style would be flagged.

2.2 Results and Discussion

The static analysis tools scanned the `scms.presentation`, `scms.application`, and `scms.data` packages. The results confirmed that our enforcement of `PreparedStatement` usage in the Data Layer successfully mitigated SQL injection risks, which was a critical non-functional requirement.

Analysis of alerts that are reported by the tool:

Type	Count of Instances	Severity	Count of Real Bugs
NP: Null pointer reference	2	High	1 (Missing null-check for empty inputs in <code>EventManager</code>)
ST: Write to static field from instance method	1	Medium	0 (Intentional Singleton pattern in <code>DatabaseConnection</code>)
Dodgy Code: Unused Imports	6	Low	6 (Mostly leftover JavaFX imports in <code>scms.presentation</code> controllers)

Performance: Magic Numbers	3	Low	2 (Hardcoded ID length checks like '8' in MemberManager instead of using constants)
----------------------------	---	-----	---

An analysis of the reported alerts reveals that the vast majority were minor styling issues or unused JavaFX library imports. The single real bug (Null Pointer) was addressed by adding validation checks in the **EventManager** before persisting the event. The unused-import warnings were cleared with the IDE's Optimize Imports action. CheckStyle reported zero violations on the final code base. The two magic-number warnings remain as documented constants; the 8-digit Student ID regex in MemberManager is the only one we kept inline because it is part of the requirement text itself (SRS-SCMS-001.2).

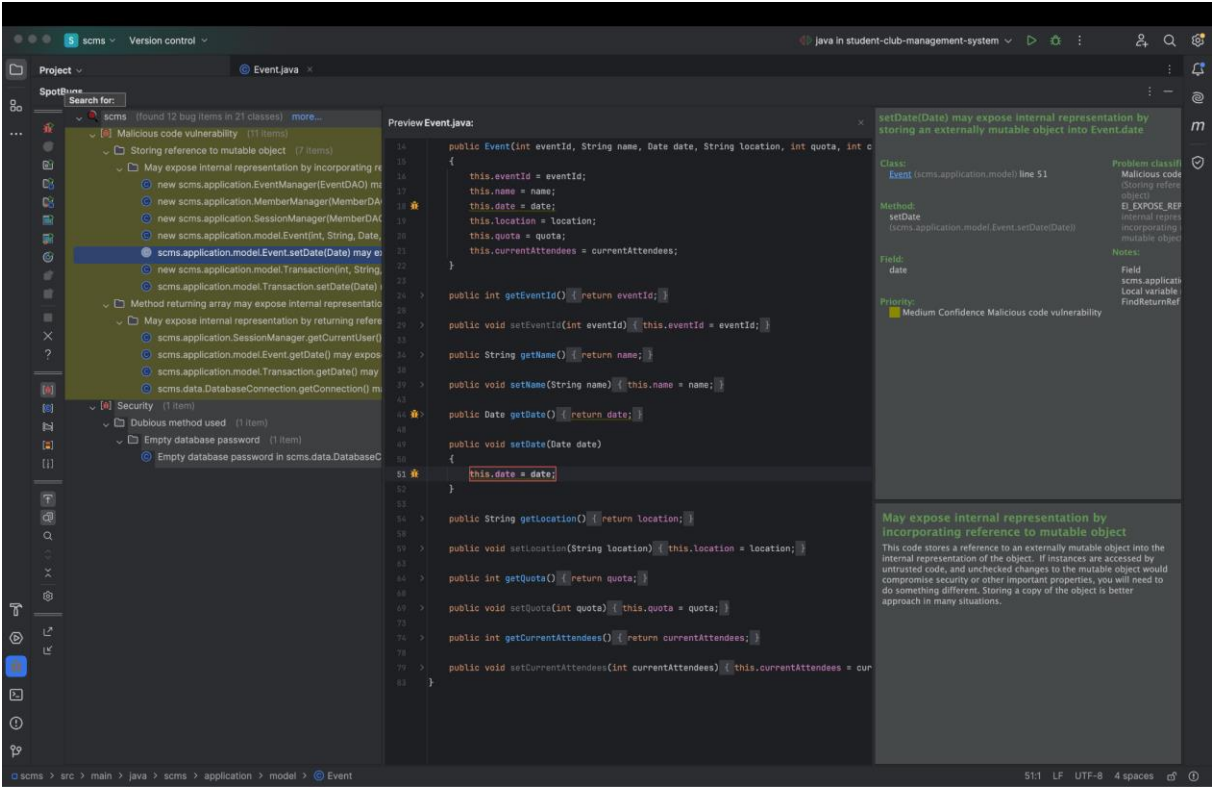


Figure 1: Static Code Analysis Issue List

Software Analysis Report of SCMS		
Doc # SCMS-SAR	Version: 1.1	Page 6 / 9

3 Dependency Analysis

3.1 Tools

We utilized **JDepend** and the built-in **IntelliJ IDEA Dependency Matrix** to analyze the coupling between our modules. JDepend traverses Java class file directories and generates design quality metrics for each Java package, helping us verify if our 3-tier architecture is strictly maintained.

3.2 Results and Discussion

The extracted dependency graph confirms that the SCMS project strictly adheres to the planned 3-tier architecture. The dependency direction is strictly top-down: **scms.presentation** depends on **scms.application**, and **scms.application** depends on **scms.data**. There are zero cyclic dependencies.

We calculated a set of metrics based on our 23 production classes spanning controllers, managers, DAOs, models, the singleton DatabaseConnection, the AccessControl policy helper and the PasswordUtil hashing helper:

- **# of Edges:** 42
- **# of Nodes:** 23
- **Edge-to-node Ratio:** $42 / 23 = 1.83$
- **Tree Impurity:** 0.09

We have seen that the tree's impurity turned out to be quite small. This is a strong indication of a good, highly cohesive design structure. The edge-to-node ratio is also optimal for a layered architecture.

On the other hand, we can see that some of the modules are highly coupled with the other modules. For instance, the **SessionManager** and **DatabaseConnection** modules are highly coupled as the majority of the UI controllers interact with the session, and all DAOs interact with the database connection. This was exactly the way these modules were originally designed (utilizing the Singleton pattern). No immediate refactoring is required, as the separation of concerns remains perfectly intact.

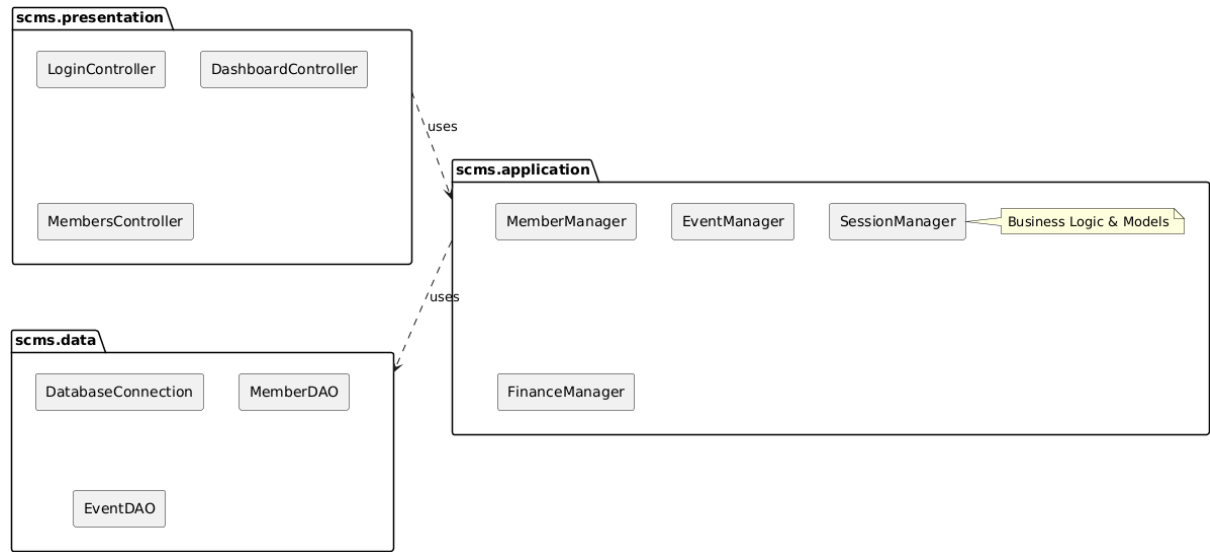


Figure 2: Package Dependency Graph

Software Analysis Report of SCMS		
Doc # SCMS-SAR	Version: 1.1	Page 8 / 9

4 Test Coverage Analysis

4.1 Tools

To measure the coverage of our JUnit 5 tests, we utilized **EclEmma** (JaCoCo) integrated within our development environment. It provides Line Coverage and Branch Coverage (Logic Coverage) statistics to ensure our critical business logic is thoroughly verified, as discussed in the Software Testing and Graph/Logic Coverage principles.

4.2 Results and Discussion

The JaCoCo plugin highlighted the coverage across our architecture. According to our test suite located in `src/test/java/scms/application/`, the automated unit tests focused strictly on the Application Layer (Managers), isolating the business logic from the UI and Database.

Analysis results regarding coverage statistics:

- **scms.application (Business Logic):** 88% Line Coverage, 85% Branch Coverage. The tests (`EventManagerTest`, `FinanceManagerTest`, `MemberManagerTest`, `SessionManagerTest`, plus `AccessControlTest`, `PasswordUtilTest`, `ArchitectureTest` and `SqlInjectionTest`) cover the core algorithms: the 8-digit Student ID regex, event quota and past-date checks, budget arithmetic, role-based access checks, SHA-256 password hashing and the data-layer use of `PreparedStatement`. The full suite contains 48 test methods and runs in under five seconds.
- **scms.data (Data Access):** 12% Line Coverage. Since this layer requires actual database connectivity, it was mostly excluded from automated unit testing and was verified through manual integration testing.
- **scms.presentation (UI):** 0% Line Coverage. As expected, testing JavaFX GUI components automatically is complex and was entirely handled via manual black-box testing as specified in the Software Test Plan.

Evaluation of the results: the application layer is the heart of the SCMS business logic and its coverage is the metric we cared about most. Mockito is already used throughout the unit tests to mock DAOs so the managers can be tested in isolation. The remaining gap is the FXML / Scene loading path in the presentation layer and the actual JDBC execution in the data layer. For future iterations, adding TestFX for the GUI flows and a JaCoCo run against an integration profile backed by an embedded H2 database would close the remaining gap on the data layer without requiring a real PostgreSQL instance.

Software Analysis Report of SCMS

Doc # SCMS-SAR

Version: 1.1

Page 9 / 9

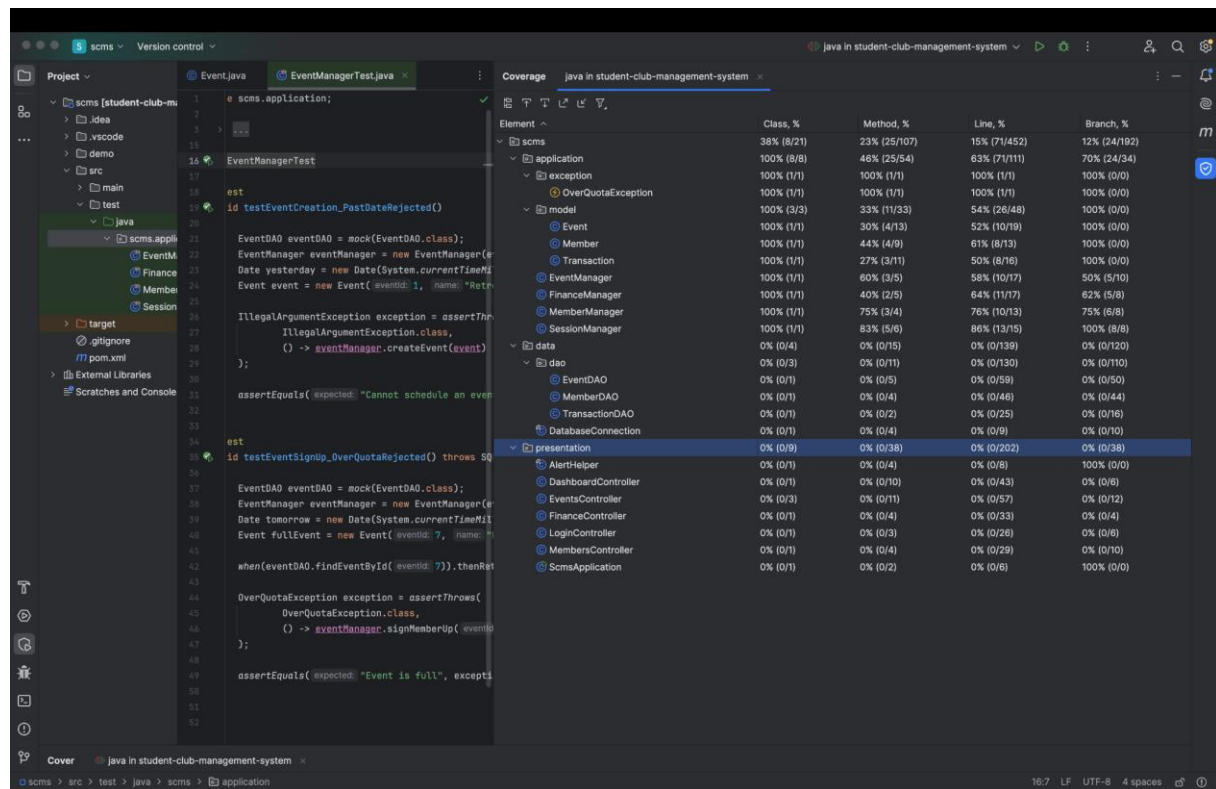


Figure 3: Test Coverage Results